DS HOMEWORK-4

1.Implement a stack using queue data structure

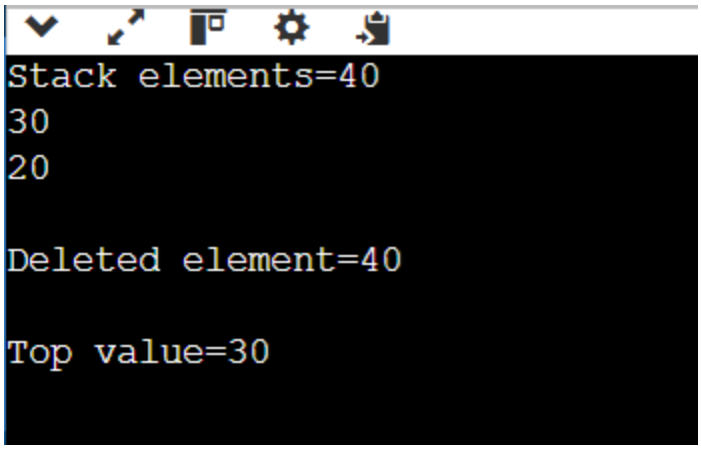
```
#include <stdio.h>
#include <stdlib.h>
#define empty_q -1
struct queue
{
   int rear;
   int front;
   int *array;
   int cap;
};
typedef struct queue *queue;
int isempty(queue q)
{
   return q->front==empty_q;
}
int isfull(queue q)
{
   return q->rear==q->cap-1;
}
void makeempty(queue q)
{
   q->front=empty_q;
   q->rear=empty_q;
}
queue create(int maxelement)
{
   queue q;
   q=(queue)malloc(sizeof(struct queue));
```

```
    if(q!=NULL)
    {
        q->array=(int *)malloc(maxelement*(sizeof(int)));
        q->cap=maxelement;
        makeempty(q);
    }
    return q;
}
void enqueue(queue q,int val)
{
    if(isfull(q))
    {
        printf("queue is full");
    }
    else if(q->rear==empty_q && q->front==empty_q)
    {
        q->front++;
        q->rear++;
        q->array[q->front]=q->array[q->rear]=val;
    }
    else
    {
        q->rear++;
        q->array[q->rear]=val;
    }
}
int dequeue(queue q)
{
    int val;
    if(isempty(q))
    {
        printf("queue is empty");
```

```
        return -1;
    }
    else if(q->front==q->rear)
    {
        val=q->array[q->front];
        makeempty(q);
    }
    else
    {

        val=q->array[q->front];
        q->front++;
    }
    return val;
}
int front(queue q)
{
    int val;
    if(isempty(q))
    {
        printf("queue is empty");
    }
    else
    {
        val=q->array[q->front];
    }
    return val;
}
void display(queue q)
{
    int i;
    if(isempty(q))
```

```
    {
        printf("queue is empty");
    }
    else if(q->rear==q->front)
    {
        printf("%d",q->array[q->rear]);
    }
    else
    {
        for(i=q->front;i<=q->rear;i++)
        {
            printf("%d \n",q->array[i]);
        }
    }
}
void dispose(queue q)
{
    if(!isempty(q))
    {
        free(q->array);
        free(q);
    }
}
queue push(queue q,int val)
{
    enqueue(q,val);
    int size=q->rear-q->front+1;
    int i;
    for(i=0;i<size-1;i++)
    {
        enqueue(q,dequeue(q));
    }
```

```
    return q;
}
int pop(queue q)
{
    int val=q->array[q->front];
    dequeue(q);
    return val;
}
int main()
{
    queue q;
    q=create(10);
    push(q,20);
    push(q,30);
    push(q,40);
    printf("Stack elements=");
    display(q);
    printf("\nDeleted element=%d\n",pop(q));
    printf("\nTop value=%d\n",front(q));
    dispose(q);
    return 0;
}
```

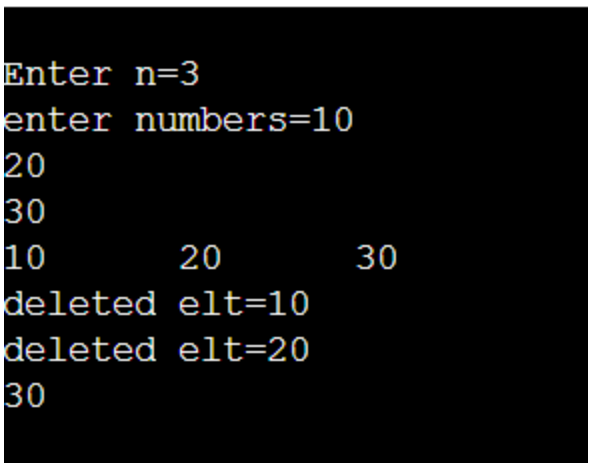


2.Implement a queue using stack data structure

```
#include<stdio.h>
#define max 100

int top=-1,stack[max];

void push(int x)
{
   top++;
   stack[top]=x;
}
int pop()
{
   int val=stack[top];
   top--;
   return val;
}

void enqueue(int x)
{
   int i=-1;
   int temp[max];
   if(top==max-1)
   {
      printf("\nqueue is full");
}
   while(top!=-1)
   {
      i++;
       temp[i]=pop();
      ;

   }
```

```
    push(x);

    while(i!=-1)
    {
        push(temp[i]);
        i--;
    }
    }


int dequeue()
{
    if(top==-1)
    {
        printf("\nqueue is empty");
        return -1;
    }
    else
    {
        int val=stack[top];
        top--;
        return val;
    }
}

void display()
{
    int i;
    for(i=top;i>=0;i--)
    {
        printf("%d\t",stack[i]);
    }
```

```
}

int main()
{
   int x;
   int n;
   printf("\nEnter n=");
   scanf("%d",&n);
   printf("enter numbers=");
   for(int i=0;i<n;i++)
   {
      scanf("%d",&x);
      enqueue(x);
   }
   display();
  printf("\ndeleted elt=%d",dequeue());
   printf("\ndeleted elt=%d\n",dequeue());
   display();
   return 0;

}
```



3.Write a c program using stack data structure to implement managing the

history of web pages visited

```
#include<stdio.h>
#include<string.h>
#define max 100
int top=-1;
char stack[max][50];
void push(char url[])
{
   if(top==max-1)
   {
      printf("\nhistory is full");
   }
   else
   {
   top++;
  strcpy(stack[top],url);
   }
}
char pop()
{
   if(top==-1)
   {
      printf("\nno previous page");
   }
   printf("\nleaving site:%s",stack[top]);
   top–;
}
void display()
{
   int i;
   printf("\nBrowsing history:");
```

```
    for(i=top;i>=0;i--)
    {
        printf("\n%s",stack[i]);
    }
    printf("\n----------\n");
}
void peek()
{
    printf("\ncurrent page=%s",stack[top]);
}
int main()
{
   push("youtube.com");
   push("instagram.com");
   push("spotify.com");
   push("facebook.com");
   display();
   peek();
   pop();
    display();
    peek();
    return 0;
}
```

4.Write a c program to sort a stack using temporary stack

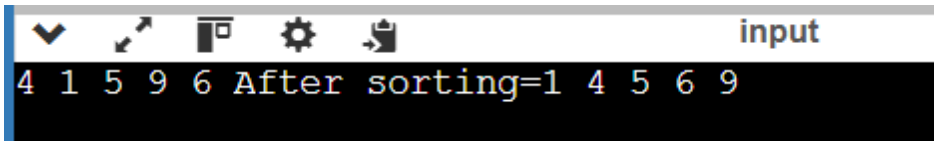
```
#include <stdio.h>
#define max 100
int top1=-1,top2=-1;
int stack1[max],stack2[max];
void push1(int val)
{
   if(top1==max-1)
   {
      printf("stack is full");
   }
   else
   {
      top1++;
      stack1[top1]=val;
   }
}
```

```
int pop1()
{
   int val;
   if(top1==-1)
   {
      printf("stack is empty");
   }
   else
   {
    val=stack1[top1];
    top1–;
   }
   return val;
}
void push2(int val)
{
   if(top2==max-1)
   {
      printf("stack is full");
   }
   else
   {
      top2++;
      stack2[top2]=val;
   }
}
int pop2()
{
   int val;
   if(top2==-1)
   {
      printf("stack is empty");
```

```
    }
    else
    {
     val=stack2[top2];
     top2--;
    }
    return val;
}
void display()
{
    int i;
    for(i=top1;i>=0;i--)
    {
        printf("%d ",stack1[i]);
    }
}

void sort()
{
    int index=0,small,n;
    n=top1;
    int temp[max];
    int val;
    while(n>0)
    {
        small=stack1[top1];
        while(top1!=-1)
        {
            val=pop1();
            if(val<small)
            {
                small=val;
```

```
            }
            push2(val);
        }
        int found=0;
        while(top2!=-1)
        {
            val=pop2();
            if(small==val && !found)
            {
                found=1;
            }
            else
            {
                push1(val);
            }
        }
        temp[index++]=small;
        n--;
    }
    for(int i=index-1;i>=0;i--)
    {
        push1(temp[i]);
    }
}
int main()
{
    push1(6);
    push1(9);
    push1(5);
    push1(1);
    push1(4);
    display();
```

```
    printf("After sorting=");
    sort();
    display();
    return 0;
}
```



5.The stock span problem is financial problem where we have a series of daily price quotes for a stock denoted by an array and the task is to calculate the span of the stocks price for all days.The span of a stocks price for i^{th} day gives the maximum number of consecutive days leading up to i^{th} day(including current day) where stocks price is less than or equal to its price on day i.

I/P=[100,80,60,70,60,75,85]

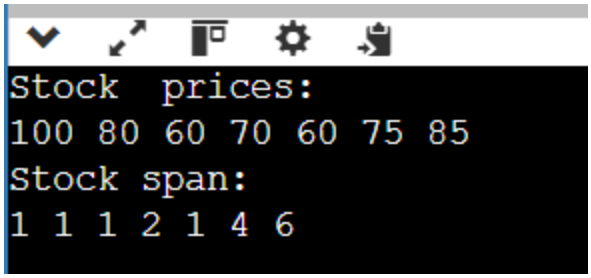
O/P=[1,1,1,2,1,4,6]

```
#include <stdio.h>
#define max 100
int top=-1;
int stack[max];
void push(int val)
{
    if(top==max-1)
    {
        printf("Stack is full\n");
    }
    else
    {
        top++;
        stack[top]=val;
    }
}
```

```
int pop()
{
   int val;
   if(top==-1)
   {
      printf("Stack is empty\n");
   }
   else
   {
     val=stack[top];
     top--;
   }
   return val;
}
void display()
{
   int i;
   if(top==-1)
   {
      printf("Stack is empty\n");
   }
   else
   {
      for(i=top;i>=0;i--)
      printf("%d ",stack[i]);
   }
}
int main()
{
   int price[max];
   int span[max];
   int n,i;
```

```
printf("enter the number of days=");
scanf("%d",&n);
printf("enter the prices=");
for(i=0;i<n;i++)
{
   scanf("%d",&price[i]);
}
//first day always has span 1
span[0]=1;
push(0);
for(i=1;i<n;i++)
{
   while(top!=-1 && price[stack[top]]<=price[i])
   {
      pop();
   }
   if(top==-1)
   {
      span[i]=i+1;
   }
   else
   {
      span[i]=i-stack[top];
   }
   push(i);
}
printf("Stock  prices:\n");
for(i=0;i<n;i++)
{
   printf("%d ",price[i]);
}
printf("\nStock span:\n");
```

```
    for(i=0;i<n;i++)
    {
        printf("%d ",span[i]);
    }
    return 0;
}
```



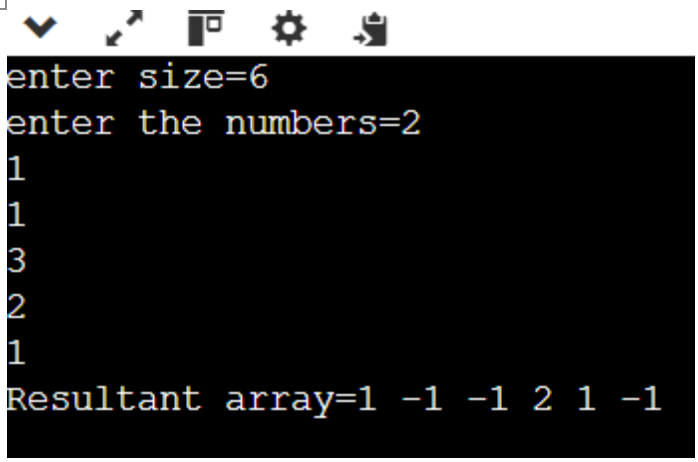
6.Given an array for each element find the nearest element to its right that has a higher frequency than the current element.If no such element exists,set value to -1.Implement using stack.

I/P=[2,1,1,3,2,1]

O/P=[1,-1,-1,2,1,-1]

```
#include <stdio.h>
void ngfe(int a[],int n)
{
    //find frequency
    int i,j,count[20];
    int result[20];
    for(i=0;i<n;i++)
    {
        count[i]=0;
    }
    for(i=0;i<n;i++)
    {
        count[a[i]]++;
    }
    result[n-1]=-1;
```

```
    for(i=0;i<n;i++)
    {
        result[i]=-1;
        for(j=i+1;j<n;j++)
        {
            if(count[a[j]]>count[a[i]])
            {
                result[i]=a[j];
                break;
            }
        }
    }
    printf("Resultant array=");
    for(i=0;i<n;i++)
    {
        printf("%d ",result[i]);
    }
}
int main()
{
    int a[20],n,i;
    printf("enter size=");
    scanf("%d",&n);
    printf("enter the numbers=");
    for(i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }
    ngfe(a,n);
    return 0;
}
```



7.Given a pattern containing only I's and D's. 'I' for increasing and 'D' for decreasing.Design an algorithm using stack to print minimum number following that pattern.Digits from 1-9 and digits can't repeat.

I/P='D'

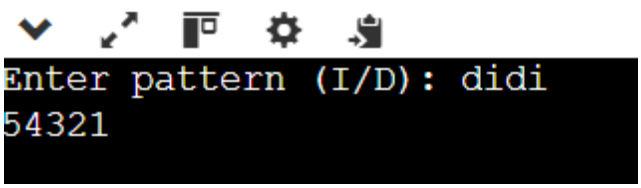
O/P=21

I/P=DIDI

O/P=21435

```
#include <stdio.h>
#define MAX 100

int stack[MAX];
int top = -1;

void push(int x)
{
   stack[++top] = x;
}

int pop()
{
   return stack[top--];
}
```

```
int main()
{
   char pattern[100];
   int i;

   printf("Enter pattern (I/D): ");
   scanf("%s", pattern);

   for(i = 0; pattern[i] != '\0'; i++)
   {
      push(i + 1);

      if(pattern[i] == 'I' )
      {
         while(top != -1)
            printf("%d", pop());
      }
   }

   push(i + 1);

   while(top != -1)
      printf("%d", pop());

   return 0;
}
```



8.Given an array of digits (values are from 0 to 9) find the minimum possible sum of two numbers formed from digits of the array.All digits of given array we used to form the two numbers use queue for implementation.

I/P=[6,8,4,5,2,3]

O/P=604(min sum formed by no's 358 and 246)

```
#include <stdio.h>
#include <stdlib.h>
#define empty_q -1
struct queue
{
   int front;
   int rear;
   int *array;
   int cap;
};

typedef struct queue *queue;
int isempty(queue q)
{
  return q->front==empty_q;
}
int isfull(queue q)
{
   return q->rear==q->cap-1;
}
void makeempty(queue q)
{
   q->front=empty_q;
   q->rear=empty_q;
}
queue create(int maxelement)
{
```

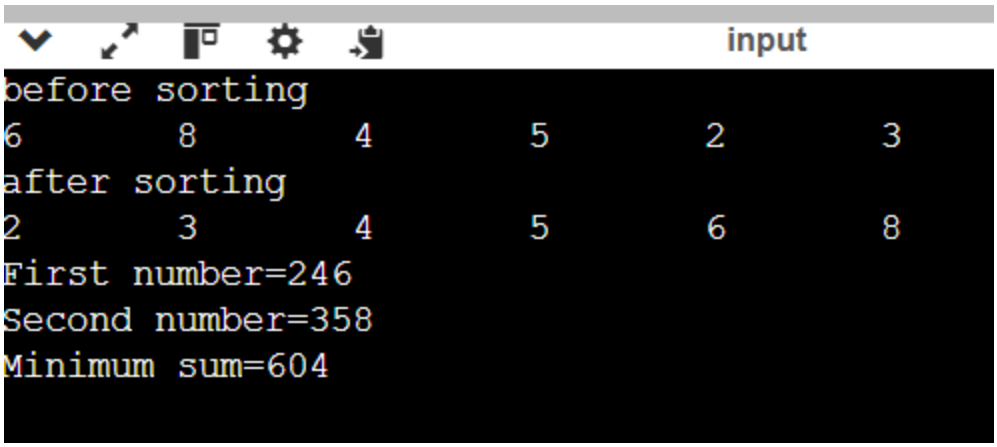
```
    queue q;
    q=(queue)malloc(sizeof(struct queue));
    if(q!=NULL)
    {
        q->array=(int*)malloc(sizeof(int)*maxelement);
        q->cap=maxelement;
        makeempty(q);
    }
    return q;
}
void enqueue(queue q,int val)
{
    if(isfull(q))
    {
        printf("queue is full\n");
    }
    else if(q->front==empty_q)
    {
        q->front++;
        q->rear++;
        q->array[q->rear]=val;
    }
    else
    {
        q->rear++;
        q->array[q->rear]=val;
    }
}
int dequeue(queue q)
{
    int val;
    if(isempty(q))
```

```
    {
        printf("queue is empty\n");
        return -1;
    }
     val=q->array[q->front];
    if(q->front==q->rear)
    {
        makeempty(q);
    }
    else
    {
        q->front++;

    }
    return val;
}
void front(queue q)
{
    if(isempty(q))
    {
        printf("queue is empty\n");
    }
    else
    {
        printf("front=%d\n",q->front);
    }
}
void display(queue q)
{
    int i;
    if(isempty(q))
    {
```

```
        printf("queue is empty\n");
    }
    else if(isfull(q))
    {
        printf("queue is full\n");
    }
    else
    {
        for(i=q->front;i<=q->rear;i++)
        {
        printf("%d\t",q->array[i]);
        }
    }
}
void sort(queue q)
{
   int i,j,temp;
   for(i=q->front;i<=q->rear;i++)
   {
      for(j=i+1;j<=q->rear;j++)
      {
         if(q->array[i]>q->array[j])
         {
            temp=q->array[i];
            q->array[i]=q->array[j];
            q->array[j]=temp;
         }
      }
   }
}
void dispose(queue q)
{
```

```
    if(q!=NULL)
    {
        free(q->array);
        free(q);
    }
}
int main()
{
    int num1=0,num2=0,sum=0,digit;
 queue q;
 q=create(10);
 enqueue(q,6);
 enqueue(q,8);
 enqueue(q,4);
 enqueue(q,5);
 enqueue(q,2);
 enqueue(q,3);
 printf("before sorting\n");
 display(q);
 sort(q);
 printf("\nafter sorting\n");
 display(q);
 int i;
 for(i=0;i<6;i++)
 {
    digit=dequeue(q);
    if(i%2==0)
    {
        num1=num1*10+digit;
    }
    else
    {
```

```
        num2=num2*10+digit;
    }
  }
 sum=num1+num2;
 printf("\nFirst number=%d",num1);
 printf("\nSecond number=%d",num2);
 printf("\nMinimum sum=%d\n",sum);
 dispose(q);
 return 0;
}
```



9.Given an array and a positive iteger ‘k’,find the first negative integer for ech window(contigious subarray) of size k.If a window does not contain a negative integer,then print 0 for that window.Implement using queue.

I/P=[-8,2,3,-6,1], k=2

O/P=[-8,0,-6,-6]

First negative integer for each window of size 2

[-8,2]=-8

[2,3]=0(no negative)

[3,-6]=-6

[-6,1]=-6

```
#include <stdio.h>
#include <stdlib.h>
```

```
#define empty_q -1
struct queue
{
   int front;
   int rear;
   int *array;
   int cap;
};

typedef struct queue *queue;
int isempty(queue q)
{
  return q->front==empty_q;
}
int isfull(queue q)
{
   return q->rear==q->cap-1;
}
void makeempty(queue q)
{
   q->front=empty_q;
   q->rear=empty_q;
}
queue create(int maxelement)
{
   queue q;
   q=(queue)malloc(sizeof(struct queue));
   if(q!=NULL)
   {
      q->array=(int*)malloc(sizeof(int)*maxelement);
      q->cap=maxelement;
      makeempty(q);
```

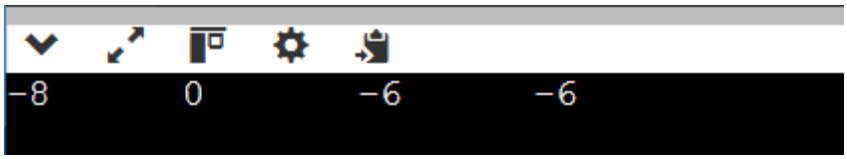
```
    }
    return q;
}
void enqueue(queue q,int val)
{
    if(isfull(q))
    {
        printf("queue is full\n");
    }
    else if(q->front==empty_q)
    {
        q->front++;
        q->rear++;
        q->array[q->rear]=val;
    }
    else
    {
        q->rear++;
        q->array[q->rear]=val;
    }
}
int dequeue(queue q)
{
    int val;
    if(isempty(q))
    {
        printf("queue is empty\n");
        return -1;
    }
     val=q->array[q->front];
    if(q->front==q->rear)
    {
```

```
        makeempty(q);
    }
    else
    {
        q->front++;

    }
    return val;
}
void front(queue q)
{
    if(isempty(q))
    {
        printf("queue is empty\n");
    }
    else
    {
        printf("front=%d\n",q->front);
    }
}
void display(queue q)
{
    int i;
    if(isempty(q))
    {
        printf("queue is empty\n");
    }
    else if(isfull(q))
    {
        printf("queue is full\n");
    }
    else
```

```
    {
        for(i=q->front;i<=q->rear;i++)
        {
        printf("%d\t",q->array[i]);
        }
    }
}
void sort(queue q)
{
  int i,j,temp;
  for(i=q->front;i<=q->rear;i++)
  {
     for(j=i+1;j<=q->rear;j++)
     {
        if(q->array[i]>q->array[j])
        {
           temp=q->array[i];
           q->array[i]=q->array[j];
           q->array[j]=temp;
        }
     }
  }
}
void dispose(queue q)
{
   if(q!=NULL)
   {
      free(q->array);
      free(q);
   }
}
int main()
```

```
{
   int a[]={-8,2,3,-6,1};
   int n=5;
   int k=2;
   int i,start;
 queue q;
 q=create(n);
 for(i=0; i<n; i++)
   {
      start = i-k+1;

      /* Remove negative numbers
        which are outside the window */
      while(!isempty(q) && q->array[q->front] < start)
      {
         dequeue(q);
      }

      /* If current element is negative,
        store its index */
      if(a[i] < 0)
      {
         enqueue(q, i);
      }

      /* Window is ready only when
        k elements are present */
      if(i >= k-1)
      {
         if(isempty(q))
         {
            printf("0\t");
```

```
        }
        else
        {
            printf("%d\t", a[q->array[q->front]]);
        }
      }
   }
 dispose(q);
 return 0;
}
```



10.Given a matrix of dimension mxn where each cell in the matrix can have values 0,1, or 2 which has the following meaning.

- 0->empty cell.
- 1->cells having fresh oranges.
- 2->cells having rotten oranges.

The task is to find the minimum stall required so that all oranges,become rotten.A rotten orange at index(i,j) can rot either fresh oranges which are its neighbours(up,down,left,right).If impossible to rot every orange then return -1.Use queue for implementation.

```
#include <stdio.h>
#include <stdlib.h>

#define empty_q -1

struct queue
{
   int front;
```

```
    int rear;
    int *array;
    int cap;
};

typedef struct queue *queue;

int isempty(queue q)
{
    return q->front == empty_q;
}

int isfull(queue q)
{
    return q->rear == q->cap - 1;
}

void makeempty(queue q)
{
    q->front = empty_q;
    q->rear = empty_q;
}

queue create(int maxelement)
{
    queue q;

    q = (queue)malloc(sizeof(struct queue));

    if(q != NULL)
    {
        q->array = (int*)malloc(sizeof(int) * maxelement);
```

```
        q->cap = maxelement;
        makeempty(q);
    }

    return q;
}

void enqueue(queue q, int val)
{
    if(isfull(q))
    {
        printf("queue is full\n");
    }
    else if(isempty(q))
    {
        q->front++;
        q->rear++;
        q->array[q->rear] = val;
    }
    else
    {
        q->rear++;
        q->array[q->rear] = val;
    }
}

int dequeue(queue q)
{
    int val;

    if(isempty(q))
    {
```

```
        return -1;
    }

    val = q->array[q->front];

    if(q->front == q->rear)
    {
        makeempty(q);
    }
    else
    {
        q->front++;
    }

    return val;
}

int main()
{
    int arr[3][5] =
    {
        {2,1,0,2,1},
        {1,0,1,2,1},
        {1,0,0,2,1}
    };

    int m = 3;
    int n = 5;

    int i,j;
    int row,col;
    int newrow,newcol;
```

```
int fresh = 0;
int time = 0;
int size;
int count;

int dr[] = {-1,1,0,0};
int dc[] = {0,0,-1,1};

queue q = create(m*n);

/* Insert all rotten oranges */
for(i=0; i<m; i++)
{
   for(j=0; j<n; j++)
   {
      if(arr[i][j] == 2)
      {
         enqueue(q, i*n+j);
      }
      else if(arr[i][j] == 1)
      {
         fresh++;
      }
   }
}

/* Process level by level */
while(!isempty(q) && fresh > 0)
{
   size = q->rear - q->front + 1;

   for(count=0; count<size; count++)
```

```
    {
        int value = dequeue(q);

        row = value / n;
        col = value % n;

        /* Check 4 neighbours */
        for(i=0; i<4; i++)
        {
            newrow = row + dr[i];
            newcol = col + dc[i];

            if(newrow >= 0 && newrow < m &&
               newcol >= 0 && newcol < n &&
               arr[newrow][newcol] == 1)
            {
                arr[newrow][newcol] = 2;
                fresh--;

                enqueue(q, newrow*n+newcol);
            }
        }
    }

    time++;
}

if(fresh > 0)
{
    printf("-1");
}
else
```

```
    {
        printf("Time=%d", time);
    }

    return 0;
}
```